TCP Sequence & ACK — Diagram Question Solution

🔑 যেকোনো TCP Diagram Question সলভ করার Master Formula

> *এই ৫টা নিয়ম মুখস্থ করলে যেকোনো TCP Seq/ACK ডায়াগ্রামের প্রশ্ন সলভ করতে পারবে।*

📐 Formula 1: Seq Number বের করা

`Next Segment` এর `Seq = Current Seq + Data Bytes sent`

মানে: আমি এখন যে Seq থেকে শুরু করলাম + কতটা ডেটা পাঠালাম = পরের Segment এর Seq।

Segment	Calculation	Seq
Seg 1	Seq = x	x
Seg 2	Seq = x + (Seg 1 এর bytes)	900
Seg 3	Seq = 900 + (Seg 2 এর bytes)	1150

📐 Formula 2: ACK Number বের করা (কোনো loss নেই)

`ACK = Last received Seq + Bytes received` = পরবর্তী যেই `Seq` টা `Receiver` আশা করছে

উদাহরণ: Seg 1 পেলে (Seq=400, 500 bytes) → ACK = 400 + 500 = **900**

📐 Formula 3: Segment হারিয়ে গেলে ACK কী হবে?

যদি `Segment N` হারিয়ে যায়: → `Receiver` সেই `Segment N` এর `Seq` আবার `ACK` করে `(Duplicate ACK)` → পরের `Segment` গুলো পেলেও একই `ACK` পাঠাতে থাকে `(Out-of-order → Buffer)` → `ACK` = হারানো `Segment` এর `Seq Number`

উদাহরণ: Seg 2 (Seq=900) হারালে:

- Seg 3 (Seq=1150) পেলেও → ACK = **900** (Duplicate)
- Retransmit পেলে → তখন সব Buffer সহ ACK = **সব Bytes এর শেষ**

📐 Formula 4: Retransmit (q) বের করা

`Timeout হলে: q = হারানো Segment এর Seq Number (একই Seq দিয়ে আবার পাঠায়)`

উদাহরণ: Seg 2 (Seq=900) হারালে → q = **900**

📐 Formula 5: Retransmit এর পরে ACK (r) বের করা

`r = Retransmitted Segment এর Seq + Retransmitted Segment এর Bytes + (Buffer এ জমা থাকা সব Bytes)`

উদাহরণ:

- Retransmit Seg 2 (Seq=900, 250 bytes)
- Buffer এ Seg 3 (Seq=1150, 50 bytes) আছে
- r = 900 + 250 + 50 = **1200**

📐 Formula 6: Port Number (Sender → Receiver এবং Reply)

`A → B: Src Port = A এর Port, Dst Port = B এর Port B → A: Src Port = B এর Port, Dst Port = A এর Port ← উল্টো!`

উদাহরণ:

- A → B: Src=7000, Dst=80
- B → A (ACK): Src=**80**, Dst=**7000**

🧠 Step-by-Step পদ্ধতি (যেকোনো প্রশ্নে এটা follow করো)

`Step 1: Seq Number বের করো → পরের Seg এর Seq দেখে আগেরটার bytes বের করো → Next Seq = Prev Seq + Bytes Step 2: Normal ACK বের করো (loss ছাড়া) → ACK = Seq + Bytes Step 3: কোন Segment Lost তা চিহ্নিত করো → Lost Segment এর পরের ACK = Duplicate (হারানো Seq আবার) Step 4: Retransmit Seq (q) বের করো → q = হারানো Segment এর Seq Step 5: Final ACK (r) বের করো → r = Retransmit Seq + তার Bytes + Buffer এর সব Bytes Step 6: Port Number → Reply তে Port উল্টে যায়`

⚡ Quick Decision Table

পরিস্থিতি	ACK কী হবে?
Segment সঠিকভাবে পৌঁছালে	ACK = Seq + Bytes (normal)
Segment হারিয়ে গেলে	ACK = হারানো Segment এর Seq (Duplicate)
Out-of-order Segment পেলে	ACK = এখনো missing Segment এর Seq
Retransmit পেলে	ACK = সব received bytes এর শেষ

📋 ডায়াগ্রাম বিশ্লেষণ (বাংলায় বুঝো)

> ***ডায়াগ্রামে কী দেখাচ্ছে:***
>
> - *Host A থেকে B-তে ৩টি Segment পাঠানো হয়েছে*
> - ***Segment 2** (Seq=900, y bytes) → ❌ **LOST***
> - *Segment 3 (Seq=1150) B-তে পৌঁছায় কিন্তু Out-of-order হওয়ায় Buffer হয়*
> - *Timeout এ A, Segment 2 আবার পাঠায়*

```
A থেকে পাঠানো: B থেকে: ──────────────────────────────────────── Seg 1: Seq=x, 500 bytes ──────────→ B পায় ✓ Seg 2: Seq=900, y bytes ─────✕──── LOST ❌ Seg 3: Seq=1150, 50 bytes ──────────→ B পায় ✓ (কিন্তু Out-of-order!) ←─── ACK = z (Seg 1 এর জন্য) ←─── ACK = p (Seg 2 missing → Duplicate ACK!) [TIMEOUT হয়!] Retransmit Seq = q ──────────────────→ B পায় ✓ ←─── ACK = r (সব পেয়েছি)
```

✅ Part i: Find x, y, z, p, q, r

> 💡 ***গুরুত্বপূর্ণ:** Seg 2 হারিয়ে গেছে। Seg 3 পৌঁছালেও B সেটা Buffer করে রাখে এবং এখনো **Seg 2 এর জন্য ACK পাঠাতে থাকে** (Duplicate ACK)।*

Finding x:

Seg 2 শুরু হয় 900 থেকে, মানে Seg 1 এর পরে 900 বাইট এসেছে।

`x + 500 = 900 ∴ x = 400`

x = 400

Finding y:

Seg 3 শুরু হয় 1150 থেকে, মানে Seg 2 এর পরে 1150 বাইট এসেছে।

`900 + y = 1150 ∴ y = 250`

y = 250

Finding z:

Seg 1 (Seq=400, 500 bytes) পাওয়ার পর B এর ACK:

`z = 400 + 500 = 900`

z = 900

Finding p:

Seg 2 (Seq=900) হারিয়ে গেছে। B এখন Seg 3 (Seq=1150) পেয়েছে। কিন্তু B এর TCP এখনো **900** নম্বরের জন্য অপেক্ষা করছে। Seg 3 Out-of-order → B Buffer করে এবং আবার **Duplicate ACK = 900** পাঠায়।

`p = 900 ← Duplicate ACK! (B still waiting for 900)`

p = 900 *(z এবং p একই হওয়াটাই সঠিক)*

Finding q:

Timeout হলে A হারানো Segment টা (Seg 2) আবার পাঠায়:

`q = 900 (Retransmit of Seg 2)`

q = 900

Finding r:

B, Seg 2 (Seq=900, 250 bytes) পায় → এখন Seg 3 (Seq=1150, 50 bytes) ও Buffer-এ আছে। B একসাথে deliver করে, তারপর ACK পাঠায়:

`r = 900 + 250 + 50 = 1200`

r = 1200

🎯 Summary Table:

Variable	Value	কেন?
x	**400**	Seg 2 starts at 900 → x = 900 − 500
y	**250**	Seg 3 starts at 1150 → y = 1150 − 900
z	**900**	ACK after Seg 1: 400 + 500 = 900
p	**900**	Seg 2 lost → Duplicate ACK (still waiting for 900)
q	**900**	Retransmit Seg 2 (same Seq)
r	**1200**	ACK after Seg 2+3 all delivered: 900+250+50

> ⚠️ ***মনে রাখো:*** *z = p = 900 একই হওয়াটা সঠিক। কারণ Seg 2 হারানোর পরে B শুধু Duplicate ACK পাঠায়।*

✅ Part ii: Segment 2 এর Source & Destination Port

> 💡 ***কনসেপ্ট:*** *একটাই TCP কানেকশনে সব Segment পাঠানো হয়। Port একই থাকে।*

	Value
Source Port	**7000**
Destination Port	**80**

✅ Part iii: B → A এর সব ACK/Segment এর Port Numbers

> 💡 ***Port Swap Rule:*** *A → B: Src=7000, Dst=80 B → A: Src=**80**, Dst=**7000** (সবসময় উল্টো!)*

B মোট **৩টি ACK** পাঠায়:

ACK	Value	Source Port	Destination Port
ACK 1	z = 900	**80**	**7000**
ACK 2	p = 900 (duplicate)	**80**	**7000**
ACK 3	r = 1200	**80**	**7000**

> *সব ACK-এই: Src = **80**, Dst = **7000***

✅ Part iv: Segment 2 NOT Lost হলে q ও r এর মান কী হবে?

> 💡 ***কনসেপ্ট:*** *যদি Seg 2 হারিয়ে না যেত, তাহলে:*
>
> - *B → Seg 2 পেয়ে ACK = 1150 পাঠাতো (z=900 এর পরে p=1150)*
> - *Seg 3 In-order পৌঁছাতো (Out-of-order না হয়ে)*
> - *Timeout হতো না*
>
> *এই পরিস্থিতিতে q এবং r:*
>
> - *q = Seg 3 এর Seq = **1150** (স্বাভাবিকভাবে পাঠানো, Retransmit না)*
> - *r = B এর ACK after Seg 3 = **1200***

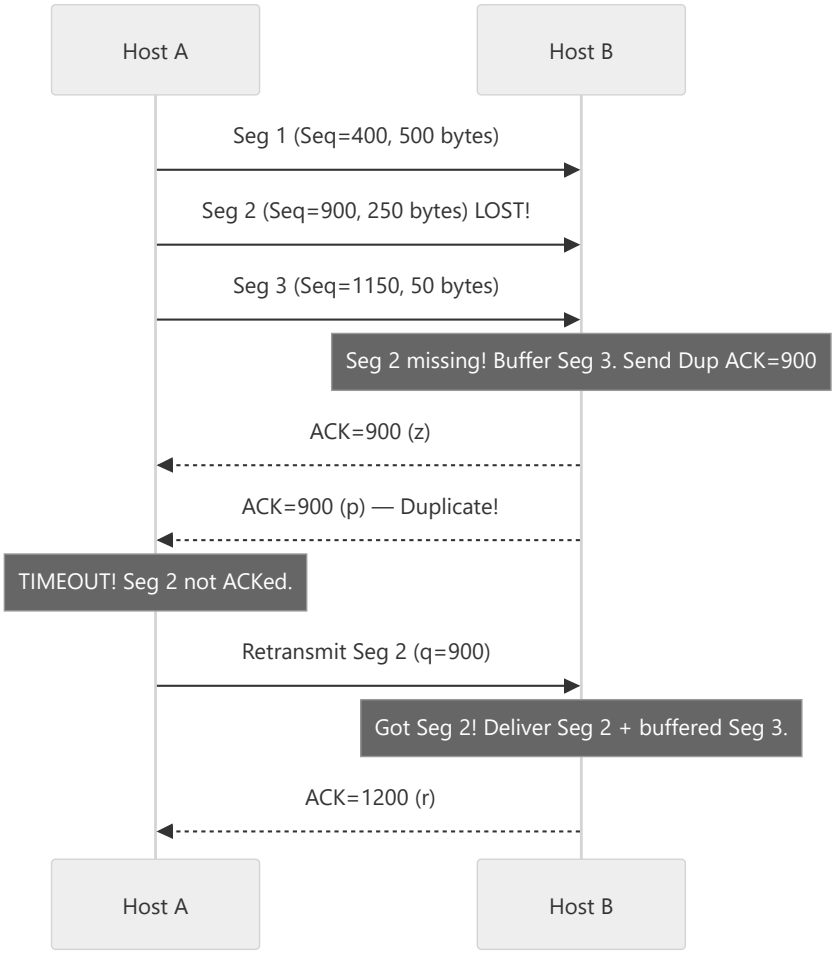
`Seg 2 NOT Lost হলে Flow: ──────────────────────────────────── A→B: Seq=400, 500 bytes → B: ACK=900 (z=900) A→B: Seq=900, 250 bytes → B: ACK=1150 (p=1150, পরিবর্তন!) A→B: Seq=1150, 50 bytes → B: ACK=1200 (এখন q=1150, r=1200) কোনো Timeout নেই, কোনো Retransmit নেই!`

	Original (Seg 2 Lost)	Seg 2 NOT Lost
z	900	900 (same)
p	900 (duplicate)	**1150** (পরিবর্তন!)
q	900 (retransmit)	**1150** (normal Seg 3 send)
r	1200	**1200** (same)

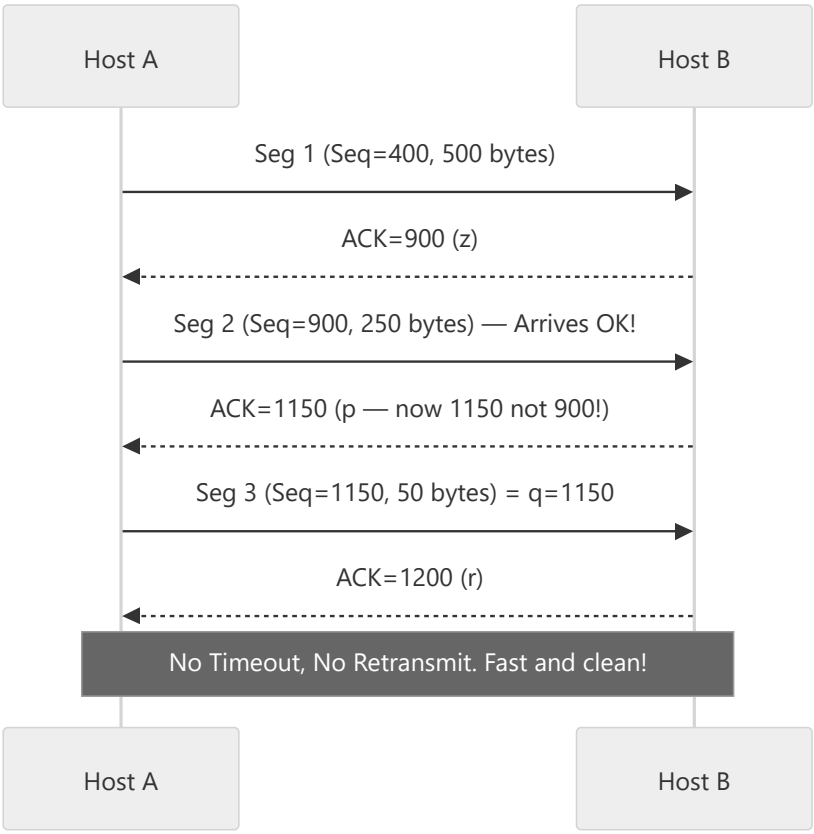
∴ q = 1150, r = 1200

📊 Timing Diagrams

Original (Segment 2 Lost):



If Segment 2 NOT Lost:



🌐 Network Layer: NAT & IP Fragmentation

🔑 IP Fragmentation — Master Formula & Solving Guide

💡 কনসেপ্ট (বাংলায়)

> ***MTU (Maximum Transmission Unit):*** *প্রতিটা নেটওয়ার্ক লিংকে একটা নির্দিষ্ট সর্বোচ্চ প্যাকেট সাইজ থাকে — এটাই MTU। যেমন Ethernet-এ সাধারণত MTU = 1500 bytes।*
>
> ***Fragmentation কেন হয়?*** *তোমার ডেটা (datagram) যদি পরবর্তী লিংকের MTU-এর চেয়ে বড় হয়, তাহলে রাউটার সেটাকে ছোট ছোট টুকরো (Fragment) করে ফেলে এবং ডেস্টিনেশনে পৌঁছে সেগুলো আবার জোড়া লাগানো হয় (Reassembly)।*
>
> ***Fields যা পরীক্ষায় জিজ্ঞেস করা হয়:***
>
> - ***ID:*** *সব Fragment-এ একই (মূল datagram-এর ID)।*
> - ***Total Length:*** *Fragment-এর Payload + Header।*
> - ***FragFlag (M-bit):*** `1` *= আরো Fragment আসবে। `0` = এটাই শেষ।*
> - ***Offset:*** *এই Fragment কোথা থেকে শুরু হয়েছে (8 বাইট unit-এ)।*

🔢 যেকোনো Fragmentation প্রশ্ন সলভ করার Step-by-Step Formula

Step	কী করবে	Formula
Step 1	Original Payload বের করো	`Total Datagram Size - Header Size`
Step 2	Max Payload বের করো	`MTU - Header Size`
Step 3	⚠️ 8-Byte Rule চেক করো	`Max Payload ÷ 8` → ভগ্নাংশ আসলে → `floor × 8`
Step 4	Fragment সংখ্যা বের করো	`ceil(Original Payload ÷ Allowed Payload)`
Step 5	প্রতি Fragment-এর Table ভরো	নিচে দেখো
Step 6	Additional Overhead	`(N - 1) × Header Size`

8-Byte Rule উদাহরণ:

- `1470 ÷ 8 = 183.75` → ভগ্নাংশ! → `floor(183.75) = 183` → `183 × 8 = 1464 bytes` ✅
- `1480 ÷ 8 = 185` → পূর্ণসংখ্যা! → সরাসরি `1480 bytes` ✅

প্রতিটি Fragment-এর জন্য:

- **Total Length** = Fragment Payload + Header
- **ID** = সবসময় একই (original datagram ID)
- **FragFlag** = `1` (শেষ ছাড়া সবার জন্য), `0` (শুধু শেষ Fragment)
- **Offset** = (আগের সব Fragment-এর মোট Payload) ÷ 8

⚡ Quick Reference — Offset Table Template

Fragment	Frag Payload	Total Length	ID	FragFlag	Offset
Frag 1	P	P + H	same	**1**	`0`
Frag 2	P	P + H	same	**1**	`P/8`
Frag 3	P	P + H	same	**1**	`2P/8`
Frag N (Last)	Remaining	Rem + H	same	**0**	`(N-1)P/8`

(P = Allowed Payload per Fragment, H = Header Size)

📘 Class Note Worked Example: Fragmentation

(তোমার ক্লাস নোটের হুবহু উদাহরণ)

দেওয়া আছে: Total Datagram = 4000 bytes, MTU = 1500 bytes, IP Header = 20 bytes

Step 1: Original Payload = 4000 - 20 = **3980 bytes**

Step 2: Max Payload = 1500 - 20 = **1480 bytes**

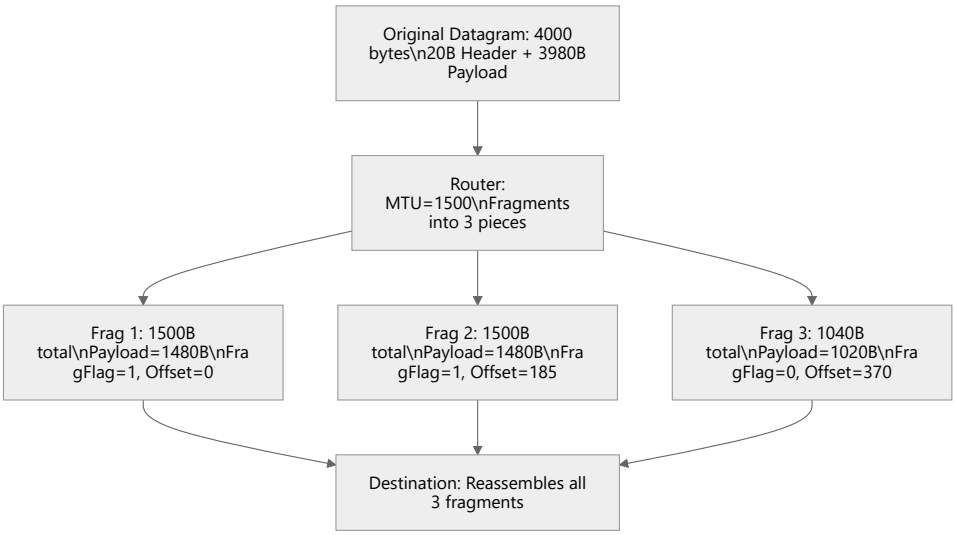
Step 3 (8-Byte Rule): 1480 ÷ 8 = **185** (পূর্ণসংখ্যা! ✅ কোনো পরিবর্তন নেই) Allowed Payload = **1480 bytes**

Step 4: 3980 ÷ 1480 = 2.68 → সুতরাং **3 টি Fragment** লাগবে

- Frag 1: 1480 bytes
- Frag 2: 1480 bytes
- Frag 3 (Remaining): 3980 - (1480 × 2) = **1020 bytes**

Step 5 — Fragmentation Table (ID = x):

Fragment	Total Length	ID	FragFlag	Offset
Frag 1	1480 + 20 = **1500**	x	**1**	0/8 = **0**
Frag 2	1480 + 20 = **1500**	x	**1**	1480/8 = **185**
Frag 3	1020 + 20 = **1040**	x	**0**	2960/8 = **370**



🔑 NAT — Master Formula & Solving Guide

💡 কনসেপ্ট (বাংলায়)

> ***NAT কেন লাগে?*** *IPv4 address মাত্র ~4.3 billion। পুরো পৃথিবীর সব ডিভাইসের জন্য যথেষ্ট না। তাই বাসায় বা অফিসে Private IP (10.x.x.x, 172.16-31.x.x, 192.168.x.x) ব্যবহার হয়। এই Private IP দিয়ে ইন্টারনেটে যাওয়া যায় না। NAT রাউটার প্রাইভেট আইপিকে নিজের পাবলিক আইপিতে কনভার্ট করে।*
>
> ***NAT কীভাবে কাজ করে?***
>
> - *Client থেকে packet এলে: Router বদলে দেয় →* ***(Private IP, Client Port) → (Public IP, NAT Port)***
> - *Server থেকে reply এলে: Router NAT Table দেখে বদলে দেয় →* ***(Public IP, NAT Port) → (Private IP, Client Port)***

🔢 যেকোনো NAT প্রশ্ন সলভ করার 4-Step Formula

Step	Packet কোথা থেকে কোথায়	Source (Src)	Destination (Dst)
Step 1	Client → Router	`Private IP : Client Port`	`Server IP : Server Port`
Step 2	Router → Server	`Public IP : NAT Port` ← **(Router বদলায়!)**	`Server IP : Server Port`
Step 3	Server → Router	`Server IP : Server Port`	`Public IP : NAT Port`
Step 4	Router → Client	`Server IP : Server Port`	`Private IP : Client Port` ← **(Router ফেরায়!)**

⚡ Quick Rule — কী পরিবর্তন হয়?

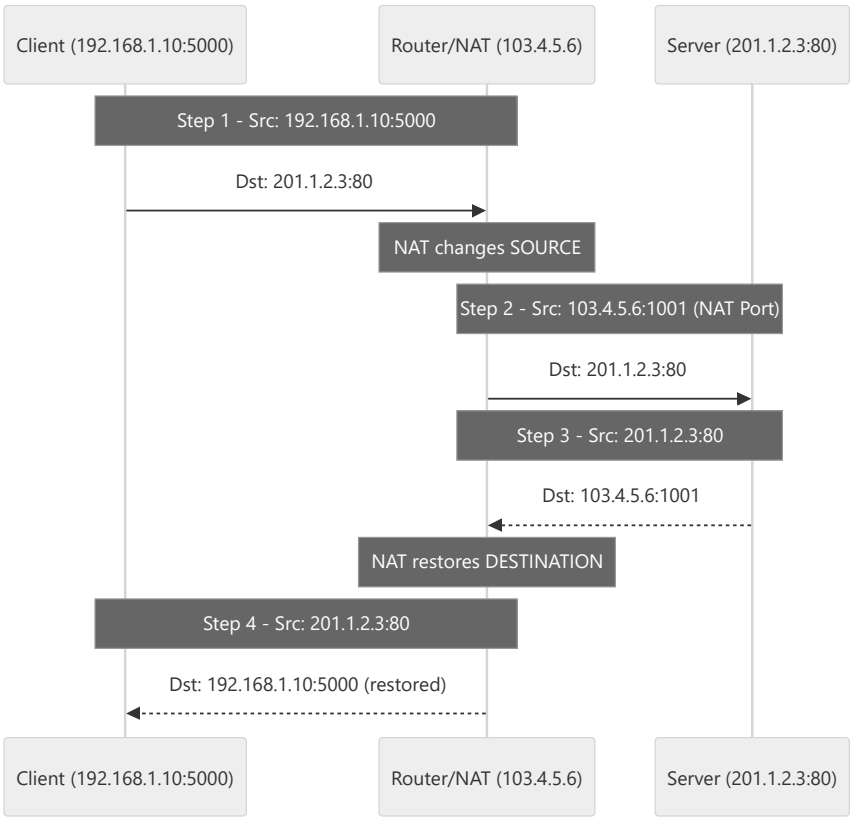
Direction	Source পরিবর্তন?	Destination পরিবর্তন?
Client → Server	**হ্যাঁ** (Private→Public)	না
Server → Client	না	**হ্যাঁ** (Public→Private)

📘 Class Note Worked Example: NAT

(তোমার ক্লাস নোটের হুবহু উদাহরণ — image 3)

Setup:

- Client 1 (Private): `192.168.1.10:5000` → Router Public IP: `103.4.5.6`
- Public Server IP: `201.1.2.3:80`



Router's NAT Table:

Inside (Private IP : Port)	Outside (Public IP : NAT Port)
192.168.1.10 : 5000	103.4.5.6 : 1001

🧠 NAT Table ভরার নিয়ম

> - ***Inside*** = *Client এর Private IP : Client Port*
> - ***Outside*** = *Router এর Public IP : একটা নতুন NAT Port (Router assign করে)*
>
> ***মনে রাখো:***
>
> - *একই Client থেকে একাধিক connection হলে → প্রতিটার জন্য* ***আলাদা NAT Port***
> - *আলাদা Client কিন্তু একই Client Port → তবুও* ***আলাদা NAT Port*** *(কারণ Private IP আলাদা)*
> - *NAT Port সাধারণত প্রশ্নে দেওয়া range থেকে Router নিজেই choose করে*

✅ Final Exam Question Solution: IP Fragmentation [Q2.c, Final 261]

Question: A datagram of 4,500 bytes arrives at a router that should be forwarded to a link with an MTU of 1,500 bytes. ID = 888. IP header = 30 bytes. Show different fragments including length, ID, fragflag, and offset values.

✅ Solution:

Step 1: Calculate Max Payload per Fragment

- Total Packet Size = 4500 bytes
- Original Payload = 4500 - 30 (Header) = 4470 bytes
- Link MTU = 1500 bytes
- Maximum Payload per Fragment = 1500 - 30 = 1470 bytes
- **8-Byte Rule Constraint:** The payload must be a multiple of 8. `1470 / 8 = 183.75` So, allowed payload = `183 × 8 = 1464 bytes`.

Step 2: Divide the Original Payload (4470 bytes)

- Fragment 1: 1464 bytes
- Fragment 2: 1464 bytes
- Fragment 3: 1464 bytes
- Fragment 4 (Remaining): `4470 - (1464 × 3) = 4470 - 4392 = 78 bytes`

Step 3: Calculate Fragmentation Table *(Total Length = Payload + Header) (Offset = Cumulative previous payload / 8) (FragFlag = 1 if more fragments follow, 0 for the last fragment)*

Fragment	Total Length (Bytes)	ID	FragFlag (M)	Offset
Fragment 1	1464 + 30 = **1494**	888	**1**	`0 / 8` = **0**
Fragment 2	1464 + 30 = **1494**	888	**1**	`1464 / 8` = **183**
Fragment 3	1464 + 30 = **1494**	888	**1**	`(1464×2) / 8` = **366**
Fragment 4	78 + 30 = **108**	888	**0**	`(1464×3) / 8` = **549**

✅ Final Exam Question Solution: NAT [Q2.d, Final 261]

Scenario Info:

- Ali's Private IP = 172.18.10.21. R1 Public IP = 201.110.50.166 (NAT ports: 7001-8000).

- Ahmad's Private IP = 192.168.70.9. R2 Public IP = 211.99.88.101 (NAT ports: 8001-9000).
- Gmail Server IP = 172.217.19.164
- Google Server IP = 216.58.218.37

i. Reply Packet from Gmail Server arriving at Router R1

Question: Ali accessing Gmail from client port 5555. What is the source & destination IP and port in the packet arriving at router R1 FROM Gmail Server?

✅ **Solution:** When Ali sends the request, R1 translates it to its Public IP and a port from its NAT range (e.g., 7001). Therefore, the reply from the Gmail Server back to R1 will be:

- **Source IP:** 172.217.19.164 (Gmail Server)
- **Source Port:** Server Port (e.g., 443 for HTTPS or 25 for SMTP)
- **Destination IP:** 201.110.50.166 (Router R1's Public IP)
- **Destination Port:** 7001 (The NAT port assigned by R1)

ii. NAT Table Entries

Question: Both Ali and Ahmad are checking Email (server port 25, client port 5500) and searching Google (server port 443, client port 6600) simultaneously. Show NAT tables for R1 & R2.

✅ **Solution:**

Router R1 NAT Table (For Ali: 172.18.10.21):

WAN Side (Outside / Global)	LAN Side (Inside / Local)
201.110.50.166 : **7001**	172.18.10.21 : 5500 *(Email)*
201.110.50.166 : **7002**	172.18.10.21 : 6600 *(Google)*

Router R2 NAT Table (For Ahmad: 192.168.70.9):

WAN Side (Outside / Global)	LAN Side (Inside / Local)
211.99.88.101 : **8001**	192.168.70.9 : 5500 *(Email)*
211.99.88.101 : **8002**	192.168.70.9 : 6600 *(Google)*

✅ Final Exam Question Solution: NAT [Q3.a, Final 251]

Scenario:

- You: Private IP 10.0.0.1, Port 3302. Your NAT Public IP 138.76.29.7 (ports 5001-5999).
- Arthur: Private IP 10.0.0.2, Port 4501. His NAT Public IP 137.91.12.44 (ports 7001-7999).
- You are sending a packet to Arthur.

✅ **Solution:** যেহেতু তুমি Arthur কে প্যাকেট পাঠাচ্ছো, তোমার প্যাকেটটি Arthur-এর Public IP এবং তার NAT পোর্ট-এ পাঠাতে হবে (ধরি Arthur-এর NAT তাকে 7001 পোর্ট দিয়েছে)।

- **Step 1 (From your host to your NAT):** Src IP: 10.0.0.1, Src Port: 3302 Dst IP: 137.91.12.44, Dst Port: 7001
- **Step 2 (Your NAT modifies and sends to Internet):** Src IP: 138.76.29.7, Src Port: 5001 *(Your NAT changes your private IP/port to its public IP and a port from its range)* Dst IP: 137.91.12.44, Dst Port: 7001
- **Step 3 (Arthur's NAT receives and modifies):** Src IP: 138.76.29.7, Src Port: 5001 Dst IP: 10.0.0.2, Dst Port: 4501 *(Arthur's NAT changes the destination back to Arthur's private IP and port)*
- **Step 4 (Arthur's Host):** Yes, his host will see the packet successfully because his NAT correctly forwards it to `10.0.0.2:4501`.

✅ Final Exam Question Solution: IP Fragmentation [Q3.b, Final 251]

Question: Packet size = 4480 bytes, Header = 20 bytes. MTU = 1500 bytes. Show length, fragflag, offset.

✅ **Solution:**

- Original Payload = 4480 - 20 = 4460 bytes.
- Link MTU = 1500 bytes. Max Payload per fragment = 1500 - 20 = 1480 bytes.
- **8-Byte Rule Check:** `1480 / 8 = 185`. (পূর্ণসংখ্যা, তাই 1480 বাইট করেই পাঠানো যাবে, কোনো প্যাডিং/পরিবর্তন লাগবে না!)

টুকরোগুলোর হিসাব:

- Frag 1: 1480 bytes
- Frag 2: 1480 bytes
- Frag 3: 1480 bytes
- Frag 4 (Remaining): 4460 - (1480 × 3) = 4460 - 4440 = 20 bytes.

Fragment	Total Length	FragFlag (M)	Offset
Frag 1	1480 + 20 = **1500**	**1**	`0 / 8` = **0**
Frag 2	1480 + 20 = **1500**	**1**	`1480 / 8` = **185**
Frag 3	1480 + 20 = **1500**	**1**	`2960 / 8` = **370**
Frag 4	20 + 20 = **40**	**0**	`4440 / 8` = **555**

✅ Final Exam Question Solution: IP Fragmentation [Q3.a, Final 252]

Question: ID = 400, Total length = 5500. MTU = 1516 bytes. Calculate length, ID, frag flag, offset. Compute additional overhead.

✅ Solution:

- Original Payload = 5500 - 20 (Header) = 5480 bytes.
- MTU = 1516 bytes. Max Payload = 1516 - 20 = 1496 bytes.
- **8-Byte Rule Check:** `1496 / 8 = 187`. (পূর্ণসংখ্যা, তাই 1496 বাইট করেই যাবে)।

টুকরোগুলোর হিসাব:

- Frag 1, 2, 3: 1496 bytes each.
- Frag 4 (Remaining): 5480 - (1496 × 3) = 5480 - 4488 = 992 bytes.

Fragment	Total Length	ID	FragFlag	Offset
Frag 1	1496 + 20 = **1516**	400	**1**	`0 / 8` = **0**
Frag 2	1496 + 20 = **1516**	400	**1**	`1496 / 8` = **187**
Frag 3	1496 + 20 = **1516**	400	**1**	`2992 / 8` = **374**
Frag 4	992 + 20 = **1012**	400	**0**	`4488 / 8` = **561**

Additional Overhead Calculation:

- আগে হেডার ছিল ১টি = ২০ বাইট।
- এখন হেডার ৪টি = ২০ × ৪ = ৮০ বাইট।
- **Additional Overhead = 80 - 20 = 60 bytes.**

✅ Final Exam Question Solution: NAT [Q3.c, Final 252]

Question: Show source & destination IP/port before and after NAT for an email service from device `10.0.0.1:242` to `193.169.1.10:25`.

✅ **Solution:** NAT Table অনুযায়ী `10.0.0.1:242` এর Public IP এবং Port হলো `120.80.30.4:3004`।

- **Before NAT (Host to Router):**
  - Source: `10.0.0.1`, Port `242`
  - Destination: `193.169.1.10`, Port `25`
- **After NAT (Router to Internet):**
  - Source: `120.80.30.4`, Port `3004`
  - Destination: `193.169.1.10`, Port `25`

✅ Final Exam Question Solution: NAT [Q2.c, Final 253]

i. Problems solved by NAT:

- **IPv4 Exhaustion:** একই Public IP ব্যবহার করে ভেতরের অনেকগুলো ডিভাইস ইন্টারনেট চালাতে পারে, ফলে IP address বাঁচে।
- **Security:** বাইরের কেউ সরাসরি ভেতরের প্রাইভেট ডিভাইসের IP জানতে পারে না বা এক্সেস করতে পারে না।

ii. 4 Steps of NAT Process (10.10.4.2:6789 to 142.250.75.142:443):

- **Step 1 (Host -> Router):** Src IP: 10.10.4.2, Port: 6789 | Dst IP: 142.250.75.142, Port: 443
- **Step 2 (Router -> Google):** Src IP: 103.109.52.2 (Router's Public IP), Port: 1234 (NAT Port) | Dst IP: 142.250.75.142, Port: 443
- **Step 3 (Google -> Router):** Src IP: 142.250.75.142, Port: 443 | Dst IP: 103.109.52.2, Port: 1234
- **Step 4 (Router -> Host):** Src IP: 142.250.75.142, Port: 443 | Dst IP: 10.10.4.2, Port: 6789

iii. Update NAT Table: *(Note: Your handwritten class note image exactly matches this answer!)*

Inside (Private IP & Port)	Outside (Public IP & NAT Port)
10.10.4.2 : 6789	103.109.52.2 : 1234
10.10.4.3 : 6789	103.109.52.2 : 1235
10.10.4.3 : 9876	103.109.52.2 : 2000

Improvement Question Solution: NAT [Q1]

Scenario Info:

- **Okabe:** Private IP 192.168.10.5. Router A Public IP: 203.112.44.71.
- **Makise:** Private IP 172.20.15.8. Router B Public IP: 219.88.63.102.
- **Mail Server:** 88.76.54.32, Port 25.
- **Web Server:** 128.83.83.20, Port 80.

(a) Okabe accesses Mail Server from client port 3512. Show NAT Process.

✅ **Solution:**

- **Step 1 (Host to Router A):**
  - **Source IP:** 192.168.10.5, **Source Port:** 3512
  - **Destination IP:** 88.76.54.32, **Destination Port:** 25
- **Step 2 (Router A to Mail Server):** *(Router changes private IP to its public IP and assigns a NAT port, e.g., 5001)*
  - **Source IP:** 203.112.44.71, **Source Port:** 5001
  - **Destination IP:** 88.76.54.32, **Destination Port:** 25
- **Step 3 (Mail Server to Router A):**
  - **Source IP:** 88.76.54.32, **Source Port:** 25
  - **Destination IP:** 203.112.44.71, **Destination Port:** 5001
- **Step 4 (Router A to Host):** *(Router changes destination back to private IP)*
  - **Source IP:** 88.76.54.32, **Source Port:** 25
  - **Destination IP:** 192.168.10.5, **Destination Port:** 3512

(b) NAT Table Entries for Router A and Router B

✅ **Solution:**

Router A NAT Table (For Okabe):

WAN Side (Public IP & NAT Port)	LAN Side (Private IP & Client Port)
203.112.44.71 : 5001	192.168.10.5 : 3512 *(Mail)*
203.112.44.71 : 5002	192.168.10.5 : 3513 *(Web)*

Router B NAT Table (For Makise):

WAN Side (Public IP & NAT Port)	LAN Side (Private IP & Client Port)
219.88.63.102 : 6001	172.20.15.8 : 3512 *(Mail)*
219.88.63.102 : 6002	172.20.15.8 : 3513 *(Web)*

Improvement Question Solution: IP Fragmentation [Q2]

Question: Total Length = `4000 + L * 8`. MTU = `1000` bytes.

Given (Student ID = 80):

- Last digit of ID = **L = 0**
- Total Length = 4000 + (0 × 8) = **4000 bytes**
- Datagram ID = **T = 80**
- MTU = **1000 bytes**, IP Header = **20 bytes** (default)

✅ **Complete Solution:**

Step 1: Original Payload

```
Original Payload = Total Length - Header = 4000 - 20 = 3980 bytes
```

Step 2: Max Payload per Fragment

```
Max Payload = MTU - Header = 1000 - 20 = 980 bytes
```

Step 3: ⚠️ 8-Byte Rule Check

`980 ÷ 8 = 122.5 ← ভগ্নাংশ! পরিবর্তন করতে হবে। floor(122.5) = 122 Allowed Payload = 122 × 8 = 976 bytes ✅`

Step 4: Fragment গণনা

`3980 ÷ 976 = 4.08 → মোট 5 টি Fragment লাগবে Frag 1: 976 bytes Frag 2: 976 bytes Frag 3: 976 bytes Frag 4: 976 bytes Frag 5 (Remaining): 3980 - (976 × 4) = 3980 - 3904 = 76 bytes`

Step 5: Fragmentation Table (ID = 80)

Fragment	Payload	Total Length	ID	FragFlag (M)	Offset
Frag 1	976	976 + 20 = **996**	**80**	**1**	`0 ÷ 8` = **0**
Frag 2	976	976 + 20 = **996**	**80**	**1**	`976 ÷ 8` = **122**
Frag 3	976	976 + 20 = **996**	**80**	**1**	`1952 ÷ 8` = **244**
Frag 4	976	976 + 20 = **996**	**80**	**1**	`2928 ÷ 8` = **366**
Frag 5	76	76 + 20 = **96**	**80**	**0**	`3904 ÷ 8` = **488**

Step 6: Additional Overhead

`আগে Header ছিল ১টি = 20 bytes এখন Header ৫টি = 20 × 5 = 100 bytes Additional Overhead = 100 - 20 = 80 bytes`

> 💡 ***চিট শিট:*** *ID = 80 → L = 0 → Total = 4000B → 5 Fragments → All ID =* ***80*** *→ Last FragFlag =* ***0*** *→ Offsets: 0, 122, 244, 366, 488*

Routing Algorithms (Network Layer)

> ***Core Question:*** *How does a router decide which path to send a packet through?* ***Answer:*** *Routing Algorithm! It finds the Least Cost path.*

---

1. Link State vs Distance Vector

> ⚠️ *Class note 5.1 had the "To whom" column* ***swapped****. Corrected below:*

Feature	Link State (LS)	Distance Vector (DV)
Concept	Global — everyone knows the full map	Decentralized — only knows neighbors
What is shared?	Full topology (all links + costs)	Own distance/routing table
To whom? ⚠️	**To ALL** (broadcast to everyone)	**Only Neighbors**
When?	Periodic + topology changes	Periodic + own table changes
Algorithm	**Dijkstra**	**Bellman-Ford**
Protocol	OSPF, IS-IS	RIP, BGP
Problem	High memory and messaging overhead	Count-to-Infinity

---

2. Link State — Dijkstra's Algorithm

Concept (বাংলায়)

Dijkstra algorithm-এ একটা router পুরো network-এর ম্যাপ জানে। সে নিজেকে **Source** ধরে বের করে বাকি সব node-এ যাওয়ার **সবচেয়ে কম খরচের (Least Cost) পথ** কোনটা।

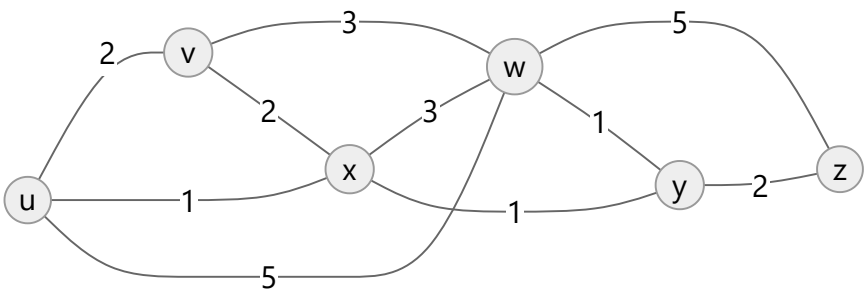
কল্পনা করো — তুমি ঢাকায়, পুরো বাংলাদেশের মানচিত্র হাতে। তুমি হিসাব করছো চট্টগ্রাম, সিলেট, রাজশাহী যেতে কোন রাস্তায় কম সময় লাগবে। এটাই Dijkstra!

Step-by-Step Rules

Step	What to do
Init	Source cost = 0. Direct neighbors = link cost. Others = infinity
Extract	Pick unvisited node with lowest D(v). Add to N'
Update	For extracted node's neighbors: `D(w) = min(D(w), D(extracted) + c(extracted, w))`
Repeat	Until all nodes are in N'
Forwarding Table	For each destination, write First Hop Link from source

Class Note Full Math — Source Node: u (5.2 image)

Network Graph:

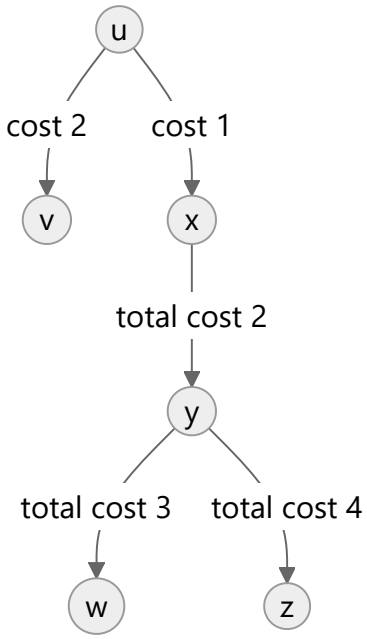


Dijkstra Table — Step by Step:

Extracted Node	D(v)	D(w)	D(x)	D(y)	D(z)
Init	inf	inf	inf	inf	inf
u (cost=0)	2, u	5, u	1, u	inf	inf
x (cost=1,u) extracted	2, u	4, x *(1+3=4 < 5)*	done	2, x *(1+1)*	inf
v (cost=2,u) extracted	done	4, x *(2+3=5 > 4, no change)*	—	2, x	inf
y (cost=2,x) extracted	—	**3, y** *(2+1=3 < 4, update!)*	—	done	**4, y** *(2+2)*
w (cost=3,y) extracted	—	done	—	—	4, y *(3+5=8 > 4, no change)*
z (cost=4,y) extracted	—	—	—	—	done

> ***Update Rule:*** `D(w) = min(old D(w), D(extracted) + c(extracted, w))` *Only the **direct neighbors** of the newly extracted node get updated!*

Least Cost Path Tree (bottom of 5.2 image)



Final Shortest Path Summary:

Destination	Shortest Cost	Full Path
v	**2**	u → v
x	**1**	u → x
y	**2**	u → x → y
w	**3**	u → x → y → w
z	**4**	u → x → y → z

Forwarding Table (5.3 image — exact match)

> ***Rule:*** *From source* `u`*, for each destination — what is the* ***FIRST HOP*** *from u?*

Destination	Path	First Hop Link
v	u → v	**(u, v)**
w	u → x → y → w	**(u, x)**
x	u → x	**(u, x)**
y	u → x → y	**(u, x)**
z	u → x → y → z	**(u, x)**

---

3. Distance Vector — Bellman-Ford Algorithm

Concept (**বাংলায়**)

DV algorithm-এ কেউ পুরো network জানে না। প্রতিটা router শুধু তার **প্রতিবেশীদের** কাছ থেকে জিজ্ঞেস করে কোথায় যেতে কত লাগে। তারপর নিজে মিলিয়ে দেখে কার মাধ্যমে সবচেয়ে সস্তা।

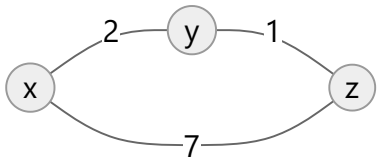
কল্পনা করো — তুমি ঢাকায়, পুরো ম্যাপ নেই। তুমি পাশের শহরকে জিজ্ঞেস করছো "ভাই, চট্টগ্রাম যেতে কত লাগে?" তারপর তোমার দূরত্ব যোগ করে বের করছো সবচেয়ে কম কোথা দিয়ে যাবে। এটাই Distance Vector!

Bellman-Ford Equation

```
D_x(y) = min over all neighbors v { c(x,v) + D_v(y) }
```

> ***Meaning:*** *Minimum cost from x to y = among all neighbors v: (x to v cost) + (v to y cost) — pick* ***minimum****.*

Worked Example — 3 Node Network



Initial DV Tables (everyone only knows direct neighbors):

	to x	to y	to z
Node x	0	2	7
Node y	2	0	1
Node z	7	1	0

After Bellman-Ford Update (x receives y and z tables):

x calculates D_x(z):

- Via y: c(x,y) + D_y(z) = 2 + 1 = **3**
- Via z: c(x,z) + D_z(z) = 7 + 0 = 7
- **min(3, 7) = 3** — Update! (was 7 before)

Updated DV Tables:

	to x	to y	to z
Node x (updated)	0	2	**3 (via y)**
Node y	2	0	1
Node z	**3 (via y)**	1	0

> *When no table changes anymore = **Convergence!** Algorithm stops.*

---

4. Count-to-Infinity Problem (DV's weakness)

When a link fails or cost increases, DV algorithm keeps circulating **wrong info** and the cost slowly increases to infinity — this is the **Count-to-Infinity** problem.

Fix: Poisoned Reverse — "If x routes to z via y, x tells y that D_x(z) = infinity" — prevents routing loops.

5. Exam Quick Revision

Dijkstra in 5 Steps:

1. **Init:** Source=0, neighbors=link cost, rest=infinity
2. **Extract:** Lowest cost unvisited node, add to N'
3. **Update:** Check if extracted node reduces neighbors' costs
4. **Repeat:** Until all nodes extracted
5. **Forwarding Table:** Write first hop link for each destination

Bellman-Ford in 3 Steps:

1. **Initial Table:** Direct neighbor costs, rest=infinity
2. **Update:** When neighbor sends table, apply BF equation
3. **Converge:** Stop when no table changes

Common Exam Questions

Question Type	What to Do
Run Dijkstra from node X	Fill the D-table step by step
Draw Least Cost Path Tree	After table done, draw final paths only
Find Forwarding Table	From tree, write first hop link for each destination
Apply Bellman-Ford for node X to Y	Calculate min{c(x,v) + D_v(y)} for all neighbors v
LS vs DV comparison	Use the comparison table above
What is Count-to-Infinity?	DV weakness — link failure causes wrong info to circulate

5 Things to Remember for 100/100

> 1. ***LS sends To ALL, DV sends Only to Neighbors*** *— most common exam mistake!*
> 2. ***Dijkstra Update:*** *Only* ***direct neighbors*** *of newly extracted node get updated*
> 3. ***Forwarding Table:*** *Write* ***FIRST HOP LINK*** *not full path — format: (source, first_neighbor)*
> 4. ***Tie Breaking:*** *Same cost for two nodes? Pick either one — final answer will be same*
> 5. ***BF Equation:*** `D_x(y) = min_v { c(x,v) + D_v(y) }` *— memorize this formula!*

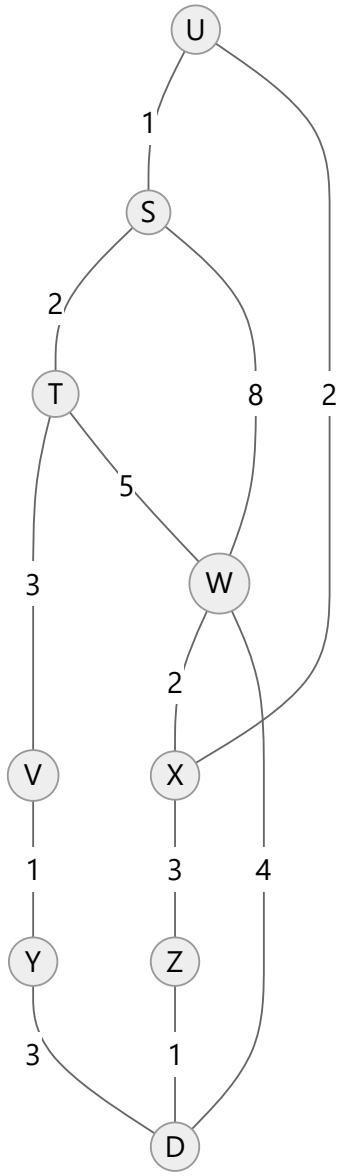
Final Exam Routing Questions : Complete Solutions (251 to 261)

> ⚠️ ***VERIFIED:*** *প্রতিটি solution exam-এর actual graph image থেকে exact weights নিয়ে solve করা হয়েছে।*

Final 251 – Q4(a) & Final 252 – Q4(a) *(Exact Same Graph!)*

Question: Given graph G = (V, E). Using Dijkstra's link-state routing algorithm, compute the least cost path from node **U** to all other nodes and show the resulting least-cost-path tree (or forwarding table) from U. [5 marks]

Graph Topology (9 nodes — from exam image):



💡 **Concept (বাংলায়):**

- Source = U। U এর সাথে সরাসরি যুক্ত: **S** (cost 1) এবং **X** (cost 2)।
- প্রতি step-এ সবচেয়ে কম cost-এর unvisited node select করো।
- Formula: `D(v) = min(D(v), D(u) + c(u, v))`
- শুধু নতুন node-এর unvisited neighbor আপডেট করবে।

Dijkstra Iteration Table (Source = U):

Step	N' (Visited)	D(S) p	D(T) p	D(V) p	D(W) p	D(X) p	D(Y) p	D(Z) p	D(D) p
Init	{U}	**1, U**	inf	inf	inf	2, U	inf	inf	inf
1	{U, **S**}	1, U	**3, S**	inf	**9, S**	2, U	inf	inf	inf
2	{U,S, **X**}	1, U	3, S	inf	**4, X**	2, U	inf	**5, X**	inf
3	{U,S,X, **T**}	1, U	3, S	**6, T**	4, X	2, U	inf	5, X	inf
4	{U,S,X,T, **W**}	1, U	3, S	6, T	4, X	2, U	inf	5, X	**8, W**
5	{U,S,X,T,W, **Z**}	1, U	3, S	6, T	4, X	2, U	inf	5, X	**6, Z**
6	{U,S,X,T,W,Z, **V**}	1, U	3, S	6, T	4, X	2, U	**7, V**	5, X	6, Z
7	{U,S,X,T,W,Z,V, **D**}	1, U	3, S	6, T	4, X	2, U	**7, V**	5, X	6, Z
8	{all, **Y**}	done	done	done	done	done	7, V	done	done

Step-by-step update explanation:

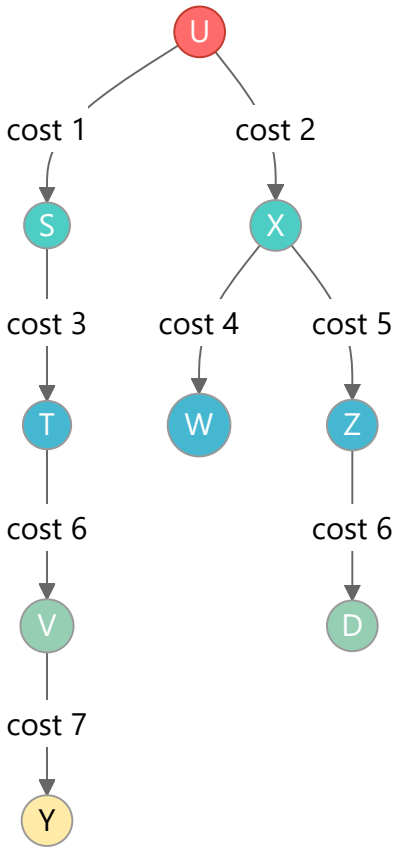
- **Step 1 (S added):** T = 1+2 = 3. W = 1+8 = 9.
- **Step 2 (X added):** W = min(9, 2+2) = **4** ✓ (updated!). Z = 2+3 = 5.
- **Step 3 (T added):** V = 3+3 = 6. W = min(4, 3+5) = 4 (no change).
- **Step 4 (W added):** D = min(∞, 4+4) = 8.
- **Step 5 (Z added):** D = min(8, 5+1) = **6** ✓ (updated!).
- **Step 6 (V added):** Y = min(∞, 6+1) = **7** ✓.
- **Step 7 (D added):** Y = min(7, 6+3) = 7 (no change since V→Y=1 gives 7, D→Y=3 gives 9).

✅ Forwarding Table for U:

Destination	Least Cost	Path	First Hop Link
S	1	U → S	**(U, S)**
X	2	U → X	**(U, X)**
T	3	U → S → T	**(U, S)**
W	4	U → X → W	**(U, X)**
Z	5	U → X → Z	**(U, X)**
V	6	U → S → T → V	**(U, S)**
D	6	U → X → Z → D	**(U, X)**
Y	7	U → S → T → V → Y	**(U, S)**

✅ Least-Cost-Path Tree from U:

> *Tree আঁকার নিয়ম: প্রতিটি node-এর parent p(.) যা, সেই অনুযায়ী সংযোগ দাও।*



Final 253 – Q2(a) & Q2(b) *(Fall 2025)*

Q2(a): Routing vs. Forwarding [2 marks]

Question: Explain why routing is a global function, while forwarding is a local function in the network layer.

💡 **Concept (বাংলায়):**

- **Routing (Global):** পুরো নেটওয়ার্কের map দেখে source থেকে destination পর্যন্ত সেরা path নির্ধারণ করা। সব router মিলে কাজ করে। যেমন Google Maps দিয়ে পুরো ঢাকা → চট্টগ্রাম route plan করা।
- **Forwarding (Local):** একটি router-এ packet আসলে, সেই router তার forwarding table দেখে প্যাকেটকে সঠিক output port দিয়ে পাঠিয়ে দেয়। শুধু একটি router-এর কাজ। যেমন মোড়ে ট্রাফিক পুলিশ গাড়ি কোন দিকে পাঠাবে তা সিদ্ধান্ত নেওয়া।

✅ **Exam Answer:**

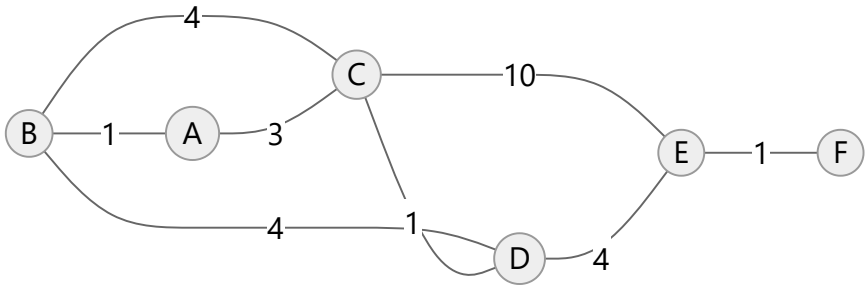
	Forwarding (Local)	Routing (Global)
Scope	Single router	Entire network
Function	Moves packet from input → output port using forwarding table	Determines end-to-end paths; builds forwarding tables
Speed	Nanoseconds (hardware)	Seconds (software, runs periodically)
Example	Router checks dest IP → sends out port 3	OSPF/Dijkstra finds shortest paths across all routers

> ***One-line exam summary:*** *"Routing **plans** the path (global); Forwarding **executes** the plan at each router (local)."*

Q2(b): Dijkstra from Node B — Final 253 *(Fall 2025)* [5 marks]

Graph (from exam image — 6 nodes: A, B, C, D, E, F):

`Exact edges from image: B — A : 1 A — C : 3 C — E : 10 B — C : 4 C — D : 1 E — F : 1 B — D : 4 D — E : 4 B — F : (no direct)`



💡 **Concept (বাংলায়):**

- Source = B। B এর neighbors: A(1), C(4), D(4)।
- সবচেয়ে কম cost = A(1)। A select করো।
- A থেকে C: D(C) = min(4, 1+3) = **4** (no change, same cost)।

Dijkstra Table (Source = B):

Step	N'	D(A) p	D(C) p	D(D) p	D(E) p	D(F) p
Init	{B}	**1, B**	4, B	4, B	inf	inf
1	{B, **A**}	1, B	**4, B**	4, B	inf	inf
2	{B,A, **C**}	1, B	4, B	**4, B**	**14, C**	inf
3	{B,A,C, **D**}	1, B	4, B	4, B	**8, D**	inf
4	{B,A,C,D, **E**}	1, B	4, B	4, B	8, D	**9, E**
5	{all, **F**}	done	done	done	done	9, E

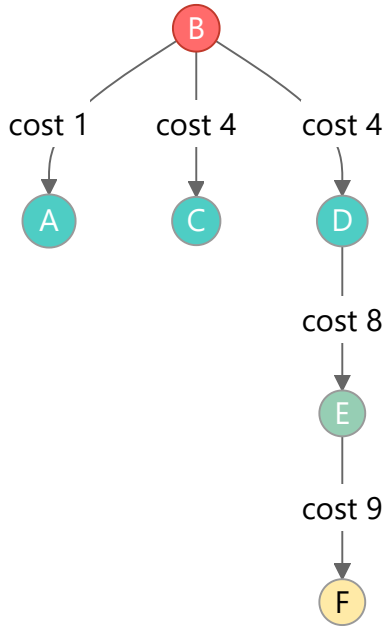
Step-by-step update explanation:

- **Step 1 (A added):** A এর neighbor C. D(C) = min(4, 1+3) = **4** (tie, parent stays B).
- **Step 2 (C added):** Tie C(4) and D(4) → take C (alphabetical). C এর neighbors: D, E.
  - D(D) = min(4, 4+1) = 4 (**no change** — B direct path is equally short).
  - D(E) = min(∞, 4+10) = **14**, parent C.
- **Step 3 (D added):** D এর neighbor E. D(E) = min(14, 4+4) = **8** ✓ (updated, parent D).
- **Step 4 (E added):** E এর neighbor F. D(F) = min(∞, 8+1) = **9** ✓ (updated, parent E).

✅ Forwarding Table for B:

Destination	Cost	Path	First Hop Link
A	1	B → A	**(B, A)**
C	4	B → C	**(B, C)**
D	4	B → D	**(B, D)**
E	8	B → D → E	**(B, D)**
F	9	B → D → E → F	**(B, D)**

✅ Least-Cost-Path Tree from B:



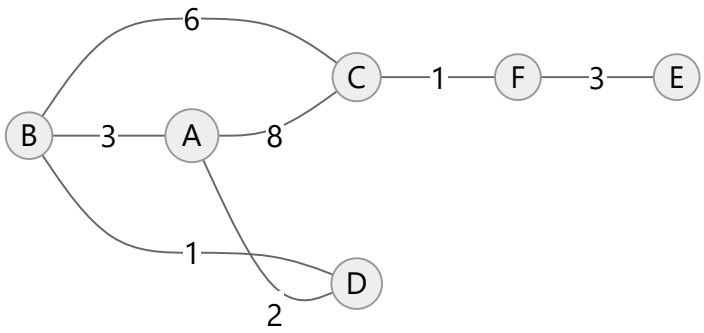
Final 261 – Q2(a), Q2(b) & Q2(c) *(Spring 2026)*

Q2(a): Routing vs. Forwarding [2 marks]

(Same answer as Final 253 Q2(a) — see above)

Q2(b): Dijkstra from Node B — Final 261 *(Spring 2026)* [5 marks]

Graph (from exam image — 6 nodes: A, B, C, D, E, F):



💡 **Concept (বাংলায়):**

- Source = B। Direct neighbors: A(3), C(6), D(1)।
- সবচেয়ে কম cost = D(1)। D select করো।
- D এর neighbors: A(2), B(visited). D(A) = min(3, 1+2) = **3** (tie! stays B).

Dijkstra Table (Source = B):

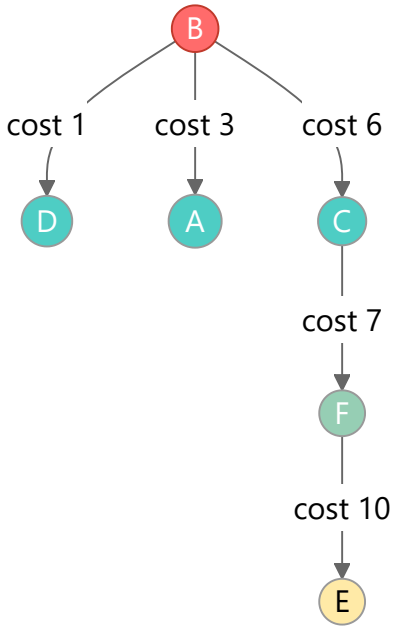
Step	N'	D(A) p	D(C) p	D(D) p	D(E) p	D(F) p
Init	{B}	**3, B**	6, B	**1, B**	inf	inf
1	{B, **D**}	**3, B**	6, B	1, B	inf	inf
2	{B,D, **A**}	3, B	**6, B**	1, B	inf	inf
3	{B,D,A, **C**}	3, B	6, B	1, B	inf	**7, C**
4	{B,D,A,C, **F**}	3, B	6, B	1, B	**10, F**	7, C
5	{all, **E**}	done	done	done	10, F	done

> ***Step 1 (D added):*** *D connects to A(2). D(A) = min(3, 1+2) = 3 (tie, no change).* ***Step 2 (A added):*** *A connects to C(8). D(C) = min(6, 3+8) = 6 (no change).* ***Step 3 (C added):*** *C connects to F(1). D(F) = min(∞, 6+1) =* ***7*** *✓.* ***Step 4 (F added):*** *F connects to E(3). D(E) = min(∞, 7+3) =* ***10*** *✓.*

✅ Forwarding Table for B:

Destination	Cost	Path	First Hop Link
D	1	B → D	**(B, D)**
A	3	B → A	**(B, A)**
C	6	B → C	**(B, C)**
F	7	B → C → F	**(B, C)**
E	10	B → C → F → E	**(B, C)**

✅ Least-Cost-Path Tree from B:



Q2(c): IP Fragmentation — Final 261 [4 marks]

Question: A datagram of **4,500 bytes** arrives. MTU = **1,500 bytes**. IP header = **30 bytes**. ID = **888**. Show fragments with Length, ID, FragFlag, Offset.

💡 **Concept (বাংলায়):**

- Total data (payload) = 4500 - 30 = **4470 bytes** (header বাদ দিয়ে)
- প্রতিটি fragment এ max data = MTU - header = 1500 - 30 = **1470 bytes**
- কিন্তু offset এর জন্য data **8-byte এর multiple** হতে হবে।
  - 1470 ÷ 8 = 183.75 → floor = 183, তাই max data per fragment = 183 × 8 = **1464 bytes**

- Total fragments = [4470 ÷ 1464] = [3.05] = **4 fragments**
- Last fragment data = 4470 - (3 × 1464) = 4470 - 4392 = **78 bytes**

✅ **Fragment Table:**

Fragment	Data (bytes)	Length = Data+Header	ID	FragFlag	Offset = Prev_data ÷ 8
1	1464	1464 + 30 = **1494**	888	**1**	**0**
2	1464	1464 + 30 = **1494**	888	**1**	1464 ÷ 8 = **183**
3	1464	1464 + 30 = **1494**	888	**1**	2928 ÷ 8 = **366**
4	78	78 + 30 = **108**	888	**0**	4392 ÷ 8 = **549**

> ✅ ***Verification:*** *1464 + 1464 + 1464 + 78 =* ***4470 bytes*** *✓* ***FragFlag = 1*** *মানে "আরও fragment আছে";* ***FragFlag = 0*** *মানে "এটাই শেষ"।*

📌 Exam Cheat Sheet — Routing Algorithm (100/100 Rules)

Rule	Detail
N' শুরু	শুধু source node দিয়ে শুরু
Initial D values	Direct neighbors = edge cost; বাকি = ∞
Node select	সবচেয়ে কম D(v) যার, সে আগে
Update formula	`D(v) = min(D(v), D(u) + c(u,v))`
কাকে update করবে	শুধু নতুন node-এর **unvisited** neighbors
Parent p(v)	D(v) update হলে p(v) = u
Forwarding Table	First Hop = source থেকে প্রথম link
Tree	প্রতিটি node কে তার parent-এর সাথে connect করো
Fragflag	Last fragment = 0, বাকি সব = 1
Offset	(Accumulated data before this fragment) ÷ 8